

Code review — Variante BAC

Runde de review pe proiectul tău, cea mai nouă sus.

Runda 3 — 2026-09-23 — sha d1f02aa

Două commit-uri (`af13cea` „final code review”, `d1f02aa` „final miercuri”) la câteva ore după ce ai primit runda 2. Ai umblat în șase din cele opt fișiere și ai adăugat varianta 9.

Poarta de compilare: `main.cpp` compilează și **apelează ceva** — `Var9sub(3,5)`. Prima dată când `main()` face efectiv o treabă. Include însă doar 2 fișiere din 9.

Fișier	Runda 2	Acum
<code>temaVar1.h</code>	✓	✓ (neatins)
<code>temaVar2.h</code>	✓	✓
<code>temaVar3.h</code>	✗	✗ o singură linie rămasă
<code>temaVar4.h</code>	✗	✓
<code>temaVar5.h</code>	✗	✗
<code>temaVar6.h</code>	✓	✓
<code>temaVar7.h</code>	✓	✓
<code>temaVar8.h</code>	✓	✓ (neatins)
<code>Var9.h</code>	—	✓ nou
<code>algortimi-utili.h</code>	✗	✗ (neatins)

De la 4 fișiere stricate la 2. Și, incluse toate împreună, **zero erori de redefinire** — erau cinci. Asta e partea bună a rundei, și e reală.

Ce ai închis

Din runda 2	Stare
B6 — terminatorul de șir	✓ rulat pe <code>baca</code> laureat : <code>baAcaA1aAuUreEaAt</code> , curat
B4 — 5 coliziuni de nume	✓ toate cinci redenumite, zero erori de redefinire
B3 — <code>cif</code> de două ori	✓ a doua e <code>cifEx4</code> acum
M1 — <code>main()</code> gol	✓ apelează <code>Var9sub(3,5)</code>

Și varianta 9, scrisă de la zero — verificată prin rulare:

Ce	Rezultat
<code>Var9sub(3,5)</code> — primele n multipli de k, descrescător	rulat: <code>15 10 5</code> ✓
<code>Var9f(1213111,1)</code>	ai scris 3 · rulat: <code>3</code> ✓
<code>strcpy / strcat</code> pe <code>abcdefg</code>	ai scris <code>abcd123efg</code> · rulat: <code>abcd123efg</code> ✓
pseudocod, n=103456	ai scris 963 · rulat: <code>963</code> ✓
pseudocod, n=1239	ai scris 1 · rulat: <code>1</code> ✓
citirea din <code>struct</code>	<code>a</code> ✓

Cinci din șase la varianta 9, și `Var9sub` e prima funcție pe care ai **rulat-o** tu, nu doar scris-o. Tabelul de la pseudocodul cu `9-c` e ținut corect pe cinci pași — aia e exact competența care se punctează la subiectul I.

Critice

B1 — ai redenumit funcțiile, dar nu și apelurile din ele

CUM E ACUM

temaVar3.h:59 — ai redenumit `f` în `Var3f`. Dar înăuntru:

```
void Var3f (int x)
{
    if(x>0)
        if(x%4==0){
            cout<<'x';
            f(x-1); }      // <- tot f
        else
        { f(x/3);          // <- tot f
          cout<<"y";
        }
}
```

Rulat, cu toate fișierele incluse:

```
Var3f(26) -> 808y
raspunsul tau (corect): xyyy
```

Și în `temaVar4.h:80`, la fel:

```
Varf(15,2) -> 1
raspunsul corect: 01111
```

Același lucru în `temaVar5.h:71` (`Var5f`, apelează `f`).

DE CE

Aici e lucrul cel mai important din toată runda, și nu e despre redenumire.

Când compilezi fișierul **singur**, primești `error: use of undeclared identifier 'f'` — corect, n-are de unde să știe ce e `f`. Dar când le incluzi pe toate, `f` **există**: `temaVar1.h` are `void f(long`

`n, int i)`, `temaVar2.h` are `void f(long n)`. Compilatorul caută un `f` care se potrivește după numărul și tipul argumentelor, îl găsește pe al altcuiva, și **compilează fără o singură eroare**.

Deci `Var3f(26)` nu mai e recursivă. La `f(x/3)` cheamă funcția variantei 2 — cea care afișează cifre — de unde `808y` în loc de `xyyy`. Nu s-a stricat nimic zgomotos: programul merge, doar răspunde altceva.

Asta e categoria de greșeală cea mai scumpă: **nu eroare de compilare, ci rezultat greșit dintr-un program care compilează**. Și a apărut *din* reparație — înainte de runda 2, coliziunea îți dădea eroare și te oprea. Acum tace.

Observă ce ai făcut **corect**, în `temaVar6.h` și `temaVar7.h`: acolo ai redenumit `transformaVar6` și ai schimbat apelul din `rezVar6`. Diferența dintre cele două situații e că acolo apelul era în **altă** funcție, deci l-ai văzut. La cele recursive, apelul e în corpul funcției pe care tocmai o redenumiai — și pe ăla nu l-ai citit ca pe „un apel”. **Într-o funcție recursivă, numele apare de două ori: în antet și înăuntru. Amândouă trebuie schimbate.**

FIX

```
// temaVar3.h – in corpul lui Var3f
Var3f(x-1);
Var3f(x/3);
// temaVar4.h – in corpul lui Varf
Varf(n,x+1);
Varf(n,x+3);
// temaVar5.h – in corpul lui Var5f
Var5f(a+1,b-2);
```

Ca să nu te mai prindă: după orice redenumire, caută numele vechi în tot fișierul (Ctrl+F în Code::Blocks) și verifică **fiecare** apariție.

B2 — `temaVar3.h:63` : ai scos bara, ai lăsat ghilimelele

CUM E ACUM

```
cout<<'x';
```

```
./temaVar3.h:63:15: error: unexpected character <U+2019>
```

Linia 67, imediat dedesubt, e reparată complet: `cout<<"y";`. Linia 63 nu. Și `temaVar5.h:73` a rămas neatinsă: `f(a+1,b-2); cout<<'*';`

DE CE

Erau **două** lucruri greșite pe linia aia, nu unul: bara `|` cu varianta în C, și ghilimelele tipografice `'` în loc de `'`. Ai scos bara — partea care se vede — și ai lăsat apostrofurile, care arată aproape identic cu cele bune pe ecran.

De-aia mesajul compilatorului îți dă **codul caracterului**, nu caracterul: `U+2019`. Apostroful corect e `U+0027`. Pe ecran diferența e de un pixel; pentru compilator sunt două caractere care n-au nicio legătură între ele. Când un mesaj de eroare îți dă un cod `U+xxxx`, ăla e semnalul că problema e un caracter care *arată* bine.

Verificarea e mecanică, nu vizuală: dacă fișierul compilează, caracterul e bun. Dacă nu, n-are rost să te uiți mai atent — nu se vede.

FIX

```
// temaVar3.h:63
cout << 'x';
// temaVar5.h:73
cout << '*';
```

B3 — `temaVar4.h:124` : ai reparat enunțul, ai lăsat codul

CUM E ACUM

În runda 2 fișierul avea enunțul **variantei 3** lipit peste cel corect, și codul rezolva varianta 3. Acum ai pus enunțul corect al variantei 4 — diagonala secundară, pe linii, cu tot cu exemplul. Codul însă e neschimbat:

```

if(i==j)
    a[i][j]=0;
else
    a[i][j]=n-j;

```

Rulat, n=4:

```

0 3 2 1      4 4 4 0
4 0 2 1      3 3 0 3
4 3 0 1  vs  2 0 2 2    <- ce cere enuntul, scris de tine deasupra
4 3 2 0      0 1 1 1

```

DE CE

Constatarea avea două jumătăți — enunțul greșit și codul greșit — și ai reparat-o pe cea de sus. Acum fișierul e **mai rău** decât era: înainte, codul se potrivea măcar cu enunțul de deasupra lui; acum enunțul spune un lucru și codul face altul, la două rânduri distanță.

Cele două condiții, una lângă alta: - diagonala **principală**: $i == j$ · valoarea vine de pe **coloană**: $n-j$ - diagonala **secundară**: $i + j == n-1$ · valoarea vine de pe **linie**: $n-i$

Pentru diagonala secundară, prima linie ($i=0$) are 0-ul pe ultima coloană ($j=3$), fiindcă $0+3 == 4-1$. Verifică pe exemplul din enunț: în linia 1 zeroul e la capăt, în linia 4 e la început. Exact ce descrie $i+j == n-1$.

FIX

```

if(i+j==n-1)
    a[i][j]=0;
else
    a[i][j]=n-i;

```

Două caractere schimbate față de ce ai. Rulează cu n=4 și compară cu exemplul pe care tocmai l-ai copiat în fișier — de-aia l-ai pus acolo.

B4 — `temaVar5.h` : ai reparat semnăturile, nu și apelurile

CUM E ACUM

```
./temaVar5.h:170:5: error: no matching function for call to 'construireVar5'
./temaVar5.h:171:5: error: no matching function for call to 'afisareVar5'
./temaVar5.h:158:14: error: use of undeclared identifier 'intj'
```

Funcțiile primesc acum parametri — exact fix-ul cerut:

```
void construireVar5(int A[11][11],int n,int m){ ... }
```

Dar apelul din `rezolvareVar5` a rămas cum era:

```
construireVar5();
afisareVar5();
```

DE CE

Același tipar ca la B1, în altă formă: ai schimbat **definiția**, nu și **locul de unde se cheamă**. Mesajul compilatorului e altul aici — `no matching function` în loc de `undeclared identifier` — și diferența merită înțeleasă: `construireVar5` **există**, deci nu e „nedeclarat”; ce nu există e o versiune a lui care se cheamă **fără argumente**. Compilatorul îți spune: știu numele, dar nu în forma asta.

Și `intj=1` de la linia 158 e tot acolo — tasta lipsă dintre `int` și `j`. Era în runda 2, la aceeași constatare.

FIX

```
void rezolvareVar5()
{
    int A[11][11], n, m;
    cin >> n >> m;
    construireVar5(A, n, m);
    afisareVar5(A, n, m);
}
```

Și la linia 158: `for (int j=1; j<=m; j++)`, iar la 159 `cout << A[i][j] << " ";` (sunt doi `<`, nu unul — cu unul singur e comparație, nu afișare).

B5 — Var9.h:57 : echivalența logică, răspunsul e **b**

CUM E ACUM

```
((a>3) && (a<15)) || (a!=b)
b. !((a<=3) || (a>=15)) || (a!=b)
d. !(a<3 || a>15) && (a!=b)
```

Ai scris **d**. Rulat pe toate perechile (a,b) din [-5,20]: varianta **b** nu diferă în niciun caz (0 din 676), varianta **d** diferă în 336.

Cel mai scurt contraexemplu: **a=2, b=5**.

```
expresia din enunt = 1      (pentru ca a!=b)
varianta d         = 0
varianta b         = 1
```

DE CE

Două motive separate pentru care **d** nu poate fi răspunsul, și e util să le vezi pe amândouă.

Primul, cel care se vede fără calcul: expresia din enunț are `||` la mijloc, **d** are `&&`. Un **sau** e adevărat când **măcar una** din părți e adevărată; un **și** cere **amândouă**. La a=2, b=5 partea stângă e falsă și partea dreaptă adevărată — **sau** dă 1, **și** dă 0. Când cauți o expresie echivalentă, **operatorul principal se păstrează**; dacă se schimbă, aproape sigur nu e ea.

Al doilea: `!(a<3 || a>15)` nu e negația lui `(a>3)&&(a<15)`. Prin De Morgan, negația lui `a<=3 || a>=15` este `a>3 && a<15` — exact partea stângă din enunț. De aia varianta **b** e identică: e aceeași condiție, scrisă negat. Varianta **d** are `<3` și `>15` în loc de `<=3` și `>=15`, deci și capetele intervalului sunt greșite, nu doar operatorul.

Ai o unealtă pentru itemii ăștia, și o folosești deja la recursivitate: **caută un singur contraexemplu**. Nu demonstra echivalența — găsește o pereche de valori pentru care cele două dau altceva. Dacă ai găsit-o, ai eliminat varianta în zece secunde.

FIX

raspuns b

🟡 Importante

M1 — `temaVar5.h:84 : 7****` are patru stele, răspunsul are cinci. Mecanismul l-ai prins — ai adăugat rândul `8<=7 nu cout<<b==7` și ai pus `7` -ul în față, exact cum trebuie. Dar tabelul tău de deasupra are acum **cinci** rânduri cu `*`, iar linia de răspuns are patru. **Rulat: `7*****`**. E a doua oară pe același exercițiu când numărătoarea finală nu se potrivește cu propriul tău tabel: numără stelele **din tabel**, nu din memorie.

M2 — `main.cpp:2-3 : include 2 fișiere din 9. Var9.h și temaVar2.h`. De-aia n-ai văzut niciuna din erorile de la B1 — `temaVar3.h`, `temaVar4.h` și `temaVar5.h` tot nu sunt compilate de nimeni. Adaugă-le pe toate; două dintre ele vor da erori, și ăla e scopul.

M3 — `temaVar2.h:197 : Var2f e tot goală.`

warning: non-void function does not return a value

Ai redenumit-o din `f` în `Var2f` — coliziunea s-a închis — dar corpul e tot `{ }`. E cerința 4a de la varianta 2 (suma exponenților din descompunerea în factori primi). Un `int` fără `return` întoarce gunoi fără să se plângă la rulare. Același warning la `rezVar6` (`temaVar6.h:96`) și `rezVar7` (`temaVar7.h:59`): declarate `int`, nu returnează nimic — ar trebui `void`.

M4 — `Var9BAC.TXT` e un fișier gol, comis. Zero octeți. Dacă e pentru cerința 3 de la varianta 9, pune datele din exemplu în el acum, cât știi ce format trebuie; dacă nu-ți trebuie, șterge-l.

M5 — `algortimi-utili.h` e neatins și tot nu compilează. Parametrii `v` și `n` redeclarați în corp (liniile 50–51), `int n;` neinițializat în `alg()`, garda cu cratimă. Semnalat în runda 2. Un fișier numit „algoritmi utili” care nu compilează e ultimul loc unde vrei să ai emoții.

Cleanup

C1 — `temaVar1.h:9` : răspunsul e tot `B` . Iar în `Var9.h:16` ai scris `d` . E același exercițiu, cuvânt cu cuvânt — intervalul `(5,8]` , aceleași patru variante. În fișierul nou l-ai rezolvat corect și ți-ai notat și condiția (`5<x<=8`). În cel vechi a rămas `raspuns B` . Deci nu e o chestie pe care n-o știi: e o linie pe care n-ai mai deschis-o. Semnalată de două runde (B3 în runda 1, C2 în runda 2).

C2 — `temaVar1.h` și `temaVar8.h` n-au fost atinse deloc. Rămân de acolo: garda de includere lipsă, `int A[100][100]` pentru `n,m ≤ 10` , `minim` cu o ramură în plus, `bac1.txt` în loc de `bac.txt` , citirea lui `m` înaintea lui `n` , tabelul lui `f(3)` fără rând de rezultat (`2` și rând nou, apoi `1 2 3`).

C3 — `strlen()` în condiția buclei, tot în trei locuri: `temaVar2.h:68` , `temaVar6.h:82` , `temaVar7.h:46` .

C4 — `bin/` , `obj/` comise, `.gitignore` tot nu există. A treia rundă.

C5 — `temaVar3.h:226–244` : al doilea enunț al lui `cif` a rămas dublat. Ai redenumit funcția în `cifEx4` și fișierul compilează — bine. Dar ai acum două funcții identice și două copii ale aceluiași enunț. Șterge-le pe cele de jos.

C6 — cerințe rămase doar ca enunț: ultimul cuvânt generat prin backtracking (V2 — răspunsul `eded` ți l-am dat în runda 1), extraprim b), palindromul (V3), eliminarea literelor mici (V4), interschimbarea în zone (V7), interclasarea cu paritate alternantă (V8), cerința 3 de la V9.

C7 — commit-urile sunt tot semnate `KERNEL0S-PC\Administrator` .

Before / After — B1

```
// ÎNAINTE – compileaza, dar Var3f(26) afiseaza 808y
void Var3f (int x)
{
    if(x>0)
        if(x%4==0){
            cout<<'x';
            f(x-1); }
        else
        { f(x/3);
          cout<<"y";
        }
}
```

```
// DUPĂ – Var3f(26) afiseaza xyyy
void Var3f (int x)
{
    if(x>0)
        if(x%4==0){
            cout<<'x';
            Var3f(x-1); }
        else
        { Var3f(x/3);
          cout<<"y";
        }
}
```

Q&A — trei întrebări

1. `Var3f(26)` afișează `808y` și **nu dă nicio eroare de compilare**. De unde vine cifra `8` ? Numește fișierul și funcția care o scrie.
2. La B4, compilatorul spune `no matching function for call to 'construireVar5'`, nu `undeclared identifier`. Ce înseamnă diferența — ce știe compilatorul în primul caz și nu știe în al

doilea?

3. În `Var9.h` ai răspuns `d` la echivalență. la perechea $a=2$, $b=5$ și calculează pe hârtie cele trei expresii — cea din enunț, `b` și `d`. Care e primul lucru din expresia `d` care ar fi trebuit să-ți atragă atenția, înainte de orice calcul?

Ce urmează

1. **B2** — două linii cu ghilimele. Apoi include **toate** fișierele în `main.cpp`.
2. **B1** — apelurile din cele trei funcții recursive. După B2 le vezi ca erori; acum sunt tăcute.
3. **B4** — apelurile din `rezolvareVar5` + `intj`.
4. **B3** — două caractere în condiția matricei, apoi rulează cu $n=4$.
5. **B5, C1** — două litere.

Ce s-a schimbat față de runda trecută: **ai reacționat în câteva ore și ai umblat în șase fișiere.**

Numărul de fișiere stricate a scăzut de la 4 la 2, coliziunile de nume s-au închis toate, și varianta 9 e cea mai curată bucată de lucru din tot proiectul.

Ce se repetă: **repari jumătatea pe care o vezi.** Bara, dar nu ghilimelele. Antetul funcției, dar nu apelul din ea. Enunțul, dar nu codul de dedesubt. Semnătura, dar nu locul de unde se cheamă. De cinci ori în aceeași rundă, același tipar.

Regula care le acoperă pe toate cinci: **după orice reparație, recompilează și rulează.** Nu ca formalitate — ca să afli ce-ai mișcat fără să vrei. Jumătate din constatările de mai sus ți le-ar fi spus compilatorul în zece secunde, iar cealaltă jumătate ți le-ar fi spus o singură rulare.

Runda 2 — 2026-09-23 — sha 7796154

1137 de linii noi, variantele 5–8. Patru din zece fișiere nu compilau; `main()` era gol.

- ☒ **B2** — ghilimele tipografice + bara `|` · **închis în** `temaVar4.h` ; în `temaVar3.h` doar pe jumătate și în `temaVar5.h` deloc (runda 3, B2)
- ☒ **B3** — `cif` definit de două ori în `temaVar3.h` · **închis** (redenumit `cifEx4` ; a rămas enunțul dublat — runda 3, C5)
- ☒ **B4** — 5 coliziuni de nume între fișiere · **TOATE ÎNCHISE**, zero erori de redefinire (dar redenumirea a deschis B1 din runda 3)

-  **B5** — `temaVar4.h` rezolva cerința variantei 3 · **INVERSAT**: enunțul e corect acum, codul nu (runda 3, B3)
-  **B6** — `cuvantNou` fără `'\0'` · **ÎNCHIS**, rulat curat
-  **B7** — `temaVar5.h`, variabile locale folosite ca globale · semnăturile reparate, apelurile nu (runda 3, B4)
-  **B8** — `f(3,17) = 7*****` · mecanismul prins, numărătoarea tot greșită (runda 3, M1)
-  **M1** — `main()` gol · apelează ceva acum, dar include 2 fișiere din 9 (runda 3, M2)
-  **M2–M7, C1–C11** — `n` neinițializat în `rez3` · `citire3` afișează de două ori · `Var2f` fără return · `algortimi-utili.h` · `NR.TXT` partajat de 3 cerințe · litera de la `temaVar1.h:9` · `.gitignore` · identitatea git — **toate neatinse**

 **Fișierul `buletinAnalize_2010.pdf` din repo:** decizie luată, subiect închis.

Verificate prin rulare și corecte în runda 2: matricea pe coloane (V3), `3798` din `NR.TXT`, `Mare Frig Rosu` cu spații multiple, sumele parțiale `15/11/8/1`, `9+36+9=54`, prefixele `10305 1030 103 10 1`, cel mai mic divizor prim, pseudocodul cu `k` (22), `f(12345) = 42`.

Runda 1 — 2026-09-15 — sha `19f5193`

Branch `master`, 4 fișiere, 500 de linii. `temaVar3.h` și `temaVar4.h` nu compilau.

-  **B1** — ghilimele tipografice + bara `|` din foaia de examen, în `temaVar3.h:63,67` și `temaVar4.h:82,87` · **NEATINS** (runda 2, B2)
-  **B2** — `rezolvare5` sărea consoanele și nu dubla vocala · **ÎNCHIS** (logica e corectă acum; a rămas terminatorul de șir — runda 2, B6)
-  **B3** — `varianta-1.h:10`, condiția gândită corect dar bifată litera `b` în loc de `d` · **NEATINS** (runda 2, C2)
-  **B4** — șapte cerințe fără rezolvare · **2 din 7 închise** (numerele divizibile cu n din `bac.txt`  rulat, și numerele cu cel mult 2 cifre din `NR.TXT` ); una rezolvată greșit (matricea cu diagonala secundară — runda 2, B5); patru rămân (runda 2, C1)
-  **M1** — `temaVar4.h` fără `#include <iostream>` · **NEATINS**
-  **M2** — `int A[100][100]` pentru `n,m ≤ 10` · **NEATINS** (runda 2, C5)
-  **M3** — `strlen()` recalculat în condiția buclei · **NEATINS**, acum în 3 locuri (runda 2, C6)
-  **M4** — `main.cpp` nu apelează nimic · **NEATINS**, acum `main()` e complet gol (runda 2, M1)
-  **M5** — `varianta-1.h` fără gardă de includere · **NEATINS** (fișierul se numește acum `temaVar1.h` — runda 2, C3)

- **C1** — `minim` cu o ramură în plus · **NEATINS** (runda 2, C7)
- **C2** — `bac1.txt` în loc de `bac.txt` · **NEATINS** (runda 2, C8)
- **C3** — citește `m` înaintea lui `n` · **NEATINS** (runda 2, C8)
- **C4** — `bin/` și `obj/` comise, fără `.gitignore` · **NEATINS** (runda 2, C9 — și cauza lui B1)

Verificate prin rulare și corecte în runda 1: `f(121,1)` → `12345` , `f(12345)` → `5310135` , `f(26)` → `xyyy` , matricea `min(i,j)` , mijlocul segmentului, `cif(a,b)` , și itemii cu variante de răspuns de la variantele 2, 3 și 4.

